In this case, the supplied function object is *copied* into the storage belonging to the newly created thread of execution and invoked from there. It's therefore essential that the copy behave equivalently to the original, or the result may not be what's expected.

One thing to consider when passing a function object to the thread constructor is to avoid what is dubbed "C++'s most vexing parse." If you pass a temporary rather than a named variable, then the syntax can be the same as that of a function declaration, in which case the compiler interprets it as such, rather than an object definition. For example,

```
std::thread my_thread(background_task());
```

declares a function `my_thread` that takes a single parameter (of type pointer to a function taking no parameters and returning a `background_task` object) and returns a `std::thread` object, rather than launching a new thread. You can avoid this by naming your function object as shown previously, by using an extra set of parentheses, or by using the new uniform initialization syntax, for example:

```
std::thread my_thread((background_task()));      ❶
std::thread my_thread{background_task()};        ❷
```

In the first example ❶, the extra parentheses prevent interpretation as a function declaration, thus allowing `my_thread` to be declared as a variable of type `std::thread`. The second example ❷ uses the new uniform initialization syntax with braces rather than parentheses, and thus would also declare a variable.

One type of callable object that avoids this problem is a *lambda expression*. This is a new feature from C++11 which essentially allows you to write a local function, possibly capturing some local variables and avoiding the need of passing additional arguments (see section 2.2). For full details on lambda expressions, see appendix A, section A.5. The previous example can be written using a lambda expression as follows:

```
std::thread my_thread([](
    do_something();
```